Taanshu Babariya 03

11/02/26

Experiment-5: Disease Prediction using Naive Bayes and Neural Network with Comparison of Classifiers

ML LAB

PID - 246001

In [41]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd
```

Load Dataset

In [42]:
```
data = load_breast_cancer()
X = data.data
y = data.target
print("Classes:", data.target_names)
```

```
Classes: ['malignant' 'benign']
```

In [43]:
```
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
df.head()
```

Out[43]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	symm
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0

5 rows × 31 columns

In [44]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
 2   mean perimeter           569 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         569 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      569 non-null    float64
 8   mean symmetry            569 non-null    float64
 9   mean fractal dimension   569 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               569 non-null    float64
 14  smoothness error         569 non-null    float64
 15  compactness error        569 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     569 non-null    float64
 28  worst symmetry           569 non-null    float64
 29  worst fractal dimension  569 non-null    float64
 30  target                   569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB
```

In [45]:
```
df.shape
```

Out[45]: (569, 31)

Split The Data

In [46]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,y,test_size=0.3,random_state=0
)
```

Logistic Regression

In [63]:
```
lr = LogisticRegression(max_iter=5000)
lr.fit(X_train, y_train)
```

```
pred_lr = lr.predict(X_test)
print("Logistic Regression")
print(f"Accuracy: {accuracy_score(y_test, pred_lr)}")
pred_lr_train = lr.predict(X_train)
print(f"Training Accuracy: {accuracy_score(y_train, pred_lr_train)}")
```

```
Logistic Regression
Accuracy: 0.9590643274853801
Training Accuracy: 0.9597989949748744
```

Decision Tree

In [64]:
```
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
pred_dt = dt.predict(X_test)
print("Decicion Tree")
print(f"Accuracy: {accuracy_score(y_test, pred_dt)}")
pred_dt_train = dt.predict(X_train)
print(f"Training Accuracy: {accuracy_score(y_train, pred_dt_train)}")
```

```
Decicion Tree
Accuracy: 0.9122807017543859
Training Accuracy: 1.0
```

KNN

In [65]:
```
knn= KNeighborsClassifier()
knn.fit(X_train, y_train)
pred_knn = knn.predict(X_test)
print("KNN")
print(f"Accuracy: {accuracy_score(y_test, pred_knn)}")
pred_knn_train = knn.predict(X_train)
print(f"Training Accuracy: {accuracy_score(y_train, pred_knn_train)}")
```

```
KNN
Accuracy: 0.9473684210526315
Training Accuracy: 0.9396984924623115
```

Naive Bayes

In [50]:
```
NB = GaussianNB()
NB.fit(X_train, y_train)
pred_NB = NB.predict(X_test)
print("Naive Bayes")
print(f"Accuracy: {accuracy_score(y_test, pred_NB)}")
pred_NB_train = NB.predict(X_train)
print(f"Training Accuracy: {accuracy_score(y_train, pred_NB_train)}")
```

```
Naive Bayes
Accuracy: 0.9239766081871345
Training Accuracy: 0.9422110552763819
```

MLP

In [51]:
```
MLP = MLPClassifier()
MLP.fit(X_train, y_train)
pred_MLP = MLP.predict(X_test)
print("MLP")
print(f"Accuracy: {accuracy_score(y_test, pred_MLP)}")
pred_MLP_train = MLP.predict(X_train)
print(f"Training Accuracy: {accuracy_score(y_train, pred_MLP_train)}")
```

```
MLP
Accuracy: 0.9532163742690059
Training Accuracy: 0.957286432160804
```

Error

Logistic Regression

In [52]:
```
cm_lr = confusion_matrix(y_test, pred_lr)
print("Confusion Matrix")
print(cm_lr)

TN1, FP1, FN1, TP1 = cm_lr.ravel()
type1_lr = FP1 / (FP1 + TN1)
type2_lr = FN1 / (FN1+TP1)

print("FN (Type - I)", FP1)
print("FP (Type - II)", FN1)
print(f"Type 1 Error in %: {type1_lr*100}")
print(f"Type 2 Error in %: {type2_lr*100}")
```

```
Confusion Matrix
[[ 62   1]
 [  6 102]]
FN (Type - I) 1
FP (Type - II) 6
Type 1 Error in %: 1.5873015873015872
Type 2 Error in %: 5.555555555555555
```

Decision Tree

In [53]:
```
cm_dt = confusion_matrix(y_test, pred_dt)
print("Confusion Matrix")
print(cm_dt)

TN2, FP2, FN2, TP2 = cm_dt.ravel()
type1_dt = FP2 / (FP2 + TN2)
type2_dt = FN2 / (FN2+TP2)

print("FN (Type - I)", FP2)
```

```
print("FP (Type - II)", FN2)
print(f"Type 1 Error in %: {type1_dt*100}")
print(f"Type 2 Error in %: {type2_dt*100}")
```

```
Confusion Matrix
[[59  4]
 [10 98]]
FN (Type - I) 4
FP (Type - II) 10
Type 1 Error in %: 6.349206349206349
Type 2 Error in %: 9.25925925925926
```

KNN

In [54]:
```
cm_knn = confusion_matrix(y_test, pred_knn)
print("Confusion Matrix")
print(cm_knn)

TN3, FP3, FN3, TP3 = cm_knn.ravel()
type1_knn = FP3 / (FP3 + TN3)
type2_knn = FN3 / (FN3 + TP3)

print("FN (Type - I)", FP3)
print("FP (Type - II)", FN3)
print(f"Type 1 Error in %: {type1_knn*100}")
print(f"Type 2 Error in %: {type2_knn*100}")
```

```
Confusion Matrix
[[ 59   4]
 [  5 103]]
FN (Type - I) 4
FP (Type - II) 5
Type 1 Error in %: 6.349206349206349
Type 2 Error in %: 4.62962962962963
```

Naive Bayes

In [55]:
```
cm_NB = confusion_matrix(y_test, pred_NB)
print("Confusion Matrix")
print(cm_NB)

TN4, FP4, FN4, TP4 = cm_NB.ravel()
type1_NB = FP4 / (FP4 + TN4)
type2_NB = FN4 / (FN4 + TP4)

print("FN (Type - I)", FP4)
print("FP (Type - II)", FN4)
print(f"Type 1 Error in %: {type1_NB*100}")
print(f"Type 2 Error in %: {type2_NB*100}")
```

```
Confusion Matrix
[[ 57   6]
 [  7 101]]
FN (Type - I) 6
FP (Type - II) 7
Type 1 Error in %: 9.523809523809524
Type 2 Error in %: 6.481481481481481
```

Neural Network

In [56]:
```
cm_MLP = confusion_matrix(y_test, pred_MLP)
print("Confusion Matrix")
print(cm_MLP)

TN5, FP5, FN5, TP5 = cm_MLP.ravel()
type1_MLP = FP5 / (FP5 + TN5)
type2_MLP = FN5 / (FN5 + TP5)

print("FN (Type - I)", FP5)
print("FP (Type - II)", FN5)
print(f"Type 1 Error in %: {type1_MLP*100}")
print(f"Type 2 Error in %: {type2_MLP*100}")
```

```
Confusion Matrix
[[ 60   3]
 [  5 103]]
FN (Type - I) 3
FP (Type - II) 5
Type 1 Error in %: 4.761904761904762
Type 2 Error in %: 4.62962962962963
```

Precision, Recall, F1 Score, ROC AUC Score

Decision tree

In [57]:
```
precision = precision_score(y_test, pred_dt)
recall = recall_score(y_test, pred_dt)
f1 = f1_score(y_test, pred_dt)
roc = roc_auc_score(y_test, pred_dt)
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score: {f1}")
print(f"ROC AUC Score: {roc}")
```

```
Precision: 0.9607843137254902
Recall: 0.9074074074074074
F1 Score: 0.9333333333333333
ROC AUC Score: 0.921957671957672
```

Logistic Regression

In [58]:
```
precision2 = precision_score(y_test, pred_lr)
recall2 = recall_score(y_test, pred_lr)
f12 = f1_score(y_test, pred_lr)
roc2 = roc_auc_score(y_test, pred_lr)
print(f"Precision: {precision2}")
print(f"Recall: {recall2}")
print(f"F1 Score: {f12}")
print(f"ROC AUC Score: {roc}")
```

```
Precision: 0.9902912621359223
Recall: 0.9444444444444444
F1 Score: 0.966824644549763
ROC AUC Score: 0.921957671957672
```

KNN

In [59]:
```
precision3= precision_score(y_test, pred_knn)
recall3= recall_score(y_test, pred_knn)
f13= f1_score(y_test, pred_knn)
roc3 = roc_auc_score(y_test, pred_knn)

print("Precision :", precision3)
print("Recall:", recall3)
print("Fi-score :", f13)
print("ROC-AUC:", roc3)
```

```
Precision : 0.9626168224299065
Recall: 0.9537037037037037
Fi-score : 0.958139534883721
ROC-AUC: 0.9451058201058201
```

Naive Bayes

In [60]:
```
precision4= precision_score(y_test, pred_NB)
recall4= recall_score(y_test, pred_NB)
f14= f1_score(y_test, pred_NB)
roc4 = roc_auc_score(y_test, pred_NB)

print("Precision :", precision4)
print("Recall:", recall4)
print("Fi-score :", f14)
print("ROC-AUC:", roc4)
```

```
Precision : 0.9439252336448598
Recall: 0.9351851851851852
Fi-score : 0.9395348837209302
ROC-AUC: 0.919973544973545
```

Neural Network

In [61]:
```
precision5 = precision_score(y_test, pred_MLP)
recall5 = recall_score(y_test, pred_MLP)
```

```
f15 = f1_score(y_test, pred_MLP)
roc5 = roc_auc_score(y_test, pred_MLP)
print("Precision :", precision5)
print("Recall:", recall5)
print("Fi-score :", f15)
print("ROC-AUC:", roc5)
```

```
Precision : 0.9716981132075472
Recall: 0.9537037037037037
Fi-score : 0.9626168224299065
ROC-AUC: 0.953042328042328
```

In [61]: